

UNIVERSITATEA „ALEXANDRU IOAN CUZA” DIN IAȘI
FACULTATEA DE INFORMATICĂ

**Project Report: Deep Learning-Based
Optimization for the Multiple Traveling
Salesmen Problem**

Maria-Alexandra Tomuleț

Chapter 1

Project Theme and Objective

The Multiple Traveling Salesman Problem is an extension of the classical Traveling Salesman Problem, in which more than one salesman is used to visit a set of cities. In this problem, each city must be visited exactly once, and each salesman must start and finish the route at the same depot. The objective is usually to minimize the total distance travelled, but in practical situations it is also important to obtain balanced routes, so that one salesman does not receive a much longer route than the others.

The purpose of this project was to implement and evaluate two different approaches for solving mTSP instances. The first approach is based on classical heuristics and uses geometric and local search techniques to construct and improve the routes. The second approach is a learning-guided method, where a neural model is trained to provide additional information that can guide the route construction process. The goal was not to develop a state-of-the-art solver, but to compare a traditional heuristic method with an experimental method that combines machine learning with heuristic optimization.

The project started from the requirement of implementing two different methods for solving the same optimization problem. For the first method, a deterministic heuristic strategy was developed. This method uses the geometric position of the nodes in order to split them between salesmen, then constructs individual routes and improves them through local search. For the second method, several versions were tested before reaching the final implementation. The final version uses predicted edge scores and node-level penalties in order to modify the route construction process and guide the heuristic toward potentially better solutions.

The implementation also included a data processing and evaluation pipeline. Benchmark instances were loaded from TSP-based datasets and transformed into mTSP instances by using different numbers of salesmen. For each instance, the algorithms generated routes that were then validated and evaluated using several metrics, such as total route cost, longest route cost, runtime, and solution validity. These metrics made it possible to compare not only the overall efficiency of each method, but also the balance between the generated routes.

Overall, this report presents the implementation process, the improvements that were attempted during development, the experimental results obtained by the two strategies, and the conclusions that can be drawn from them. The analysis is objective and focuses on what was actually implemented and observed. The results show that the learning-guided method is competitive in some cases, especially regarding route balance, but it does not consistently outperform the classical heuristic in terms of total cost.

Chapter 2

Problem Definition and Data Preparation

The problem addressed in this project is the Multiple Traveling Salesman Problem. This problem is based on the classical Traveling Salesman Problem, but instead of using a single salesman to visit all cities, several salesmen are available. Each salesman starts from the same depot, visits a subset of the cities, and returns to the depot at the end of the route. A valid solution must ensure that every city, except the depot, is visited exactly once and that all routes start and end in the depot.

In this project, the objective was mainly to minimize the total travel cost of all routes. However, the longest route cost was also considered important, because it gives information about how balanced the solution is. A solution with a low total cost may still be less practical if one salesman receives a much longer route than the others. For this reason, the evaluation did not focus only on the total distance, but also on route balance, runtime, and solution validity.

The data used in the project was based on standard TSP benchmark instances. These instances contain a set of nodes represented by coordinates, and the distance between nodes can be computed from their positions. In order to use them for the Multiple Traveling Salesman Problem, each TSP instance was transformed into several mTSP instances by changing the number of salesmen. The first node was considered the depot, while the remaining nodes had to be assigned to the available salesmen.

The project used several benchmark instances, including small and medium-sized problems. For each base instance, multiple configurations were tested by varying the number of salesmen. This allowed the two implemented strategies to be evaluated not only on different problem sizes, but also on different route distribution scenarios. Testing the algorithms with 2, 3, 5, and 7 salesmen made it possible to observe how the methods behave when the number of routes increases.

The data preparation stage included reading the input files, extracting the node coordinates, computing the distance matrix, and creating the internal representation required by the solvers. The distance matrix was used by both strategies during route construction and improvement. This preprocessing step was important because it ensured that both methods were tested under the same conditions and used the same input data.

For the learning-guided strategy, an additional data preparation step was necessary. Since the model required training examples, a teacher solver was used to generate reference solutions. These solutions were not exact optimal solutions, but they provided useful guidance for the model. Based on the teacher results, the project generated edge-level information that could be used during training. This allowed the second strategy to learn which candidate connections were more likely to appear in better routes.

The final evaluation was based on several metrics. The first metric was the total cost of all generated routes, which measured the overall efficiency of the solution. The second metric was the longest route cost, which indicated how balanced the routes were between salesmen.

Runtime was also measured in order to compare the computational effort required by each method. Finally, each solution was validated to ensure that all cities were visited exactly once and that the routes respected the structure of the mTSP.

Chapter 3

Implementation of the First Strategy

The first strategy implemented in this project represents the classical heuristic approach used as a baseline for solving the Multiple Traveling Salesman Problem. The purpose of this strategy was to obtain valid and reasonably good solutions without using machine learning. The method is based on geometric information, constructive heuristics, and local search improvements. This made it useful both as an independent solving method and as a reference point for evaluating the second strategy.

The general idea of the first strategy is to divide the nodes between the available salesmen, build one route for each salesman, and then improve the routes through local optimization. Since the problem is based on coordinates, the geometric position of the nodes was used during the assignment stage. This helped create routes that are spatially coherent, instead of assigning nodes randomly to salesmen.

The first important step is the geometric sweep assignment. In this step, the depot is used as the central reference point, and the angle of each node relative to the depot is computed. The nodes are then sorted by angle and divided between the salesmen. This approach tries to assign geographically close nodes to the same salesman, which usually leads to more natural and compact routes. It is not guaranteed to produce an optimal assignment, but it provides a good initial structure for the solution.

After the nodes are assigned to each salesman, the individual routes are constructed using a nearest insertion heuristic. This method starts with a simple route and repeatedly inserts unvisited nodes in positions that produce the smallest cost increase. Compared to a simple nearest neighbor approach, nearest insertion is more flexible because it does not only add the next closest node at the end of the route, but searches for a better insertion position inside the current route.

The route construction process can be summarized as follows:

- the nodes are first assigned to salesmen using the geometric sweep method;
- for each salesman, a route is built using nearest insertion;
- each route starts and ends at the depot;
- all non-depot nodes must appear in exactly one route.

Once the initial routes are constructed, the algorithm applies 2-opt local search to each route. The purpose of 2-opt is to reduce unnecessary crossings and improve the internal order of the nodes. It works by selecting two edges from a route and reversing the segment between them if this reduces the route cost. This is a common improvement method for TSP-like problems and is useful because the constructive phase does not always produce the best node order.

After improving each route separately, the strategy also tries to improve the solution at the level of all salesmen. For this, inter-route relocation moves are used. This means that a node can be moved from one salesman route to another if the move decreases the overall cost. This step is important because the initial sweep assignment may not always distribute the nodes in the best possible way. By allowing nodes to move between routes, the algorithm can correct some of the limitations of the initial assignment.

The main improvement steps used in the first strategy are:

- 2-opt local search, used to improve the order of nodes inside each route;
- inter-route relocation, used to move nodes between different routes;
- final 2-opt refinement, used after relocation to improve the modified routes;
- validation, used to check that the final solution respects the mTSP constraints.

The final result of this strategy is a set of valid routes, one for each salesman. Each route starts and ends at the depot, and every non-depot node is visited exactly once. The total cost, longest route cost, and runtime are then computed for evaluation. This strategy is relatively simple, but it is reliable and produced valid solutions for the tested benchmark instances.

Chapter 4

Implementation of the Second Strategy and Improvements

The second strategy was implemented as an experimental learning-guided approach. Its purpose was to test whether machine learning could be used to guide the route construction process and obtain competitive results compared to the classical heuristic strategy. Instead of replacing the heuristic completely, the neural model was used to provide additional information that modifies how the routes are built.

The main idea of this strategy is to predict useful connections between nodes and then use these predictions during the construction of the mTSP routes. The method does not directly output the final routes. Instead, it estimates which candidate edges are more promising and also assigns penalties to nodes. These predictions are then combined with the distance information in order to guide the nearest insertion process.

A first important step in this strategy is the construction of candidate edges. Considering all possible edges between all nodes would be inefficient and unnecessary, especially because many long-distance edges are unlikely to be useful in a good route. For this reason, the method keeps only a limited number of nearby candidate neighbors for each node. This creates a sparse graph that focuses the model on more relevant connections.

For each candidate edge, several features are extracted. These features describe the geometric relationship between the two nodes and their position relative to the depot. The model can therefore learn patterns based not only on distance, but also on spatial structure. The features include information such as the coordinates of the nodes, the distance between them, their distance from the depot, the angular difference, the neighbor rank, and whether the reverse connection also exists.

The training data for the model is generated using a teacher solver. The teacher solver is not an exact optimization algorithm, but a stronger heuristic based on multiple restarts. It runs a version of the classical strategy several times with different configurations and keeps the best solution found. The edges that appear in the teacher solution are then used as reference information for training the model. In this way, the model learns from solutions that are usually better than a single basic heuristic run.

The final version of the second strategy uses two types of predictions:

- edge scores, which estimate how useful a candidate edge may be;
- node penalties, which estimate how difficult or costly a node may be during route construction.

The edge scores are used to make promising connections more attractive during route construction. The node penalties add node-level information, so the method does not only

evaluate connections between pairs of nodes, but also considers the individual importance or difficulty of each node. These predictions are combined with the original distance matrix and used by the heuristic construction process.

After the model produces the predictions, the strategy still uses a heuristic solver to generate the final routes. This means that the second strategy remains a hybrid method: the neural model guides the search, while the heuristic algorithm constructs and improves the actual solution. After route construction, the same types of local improvements are applied, including 2-opt and relocation moves between routes. This makes the comparison with the first strategy more consistent, because both methods use similar improvement mechanisms after the initial construction phase.

During development, several versions of the learning-guided strategy were tested. Earlier versions focused mainly on predicting useful edges. Later versions improved the data generation process and the way predictions were integrated into the solver. The final version added node-level penalties, which made the method more flexible because it allowed the algorithm to adjust the cost of visiting certain nodes, not only the cost of using certain edges.

The most important improvements attempted in the second strategy were:

- generating training labels using a stronger teacher solver with multiple restarts;
- using candidate edges instead of all possible edges;
- adding geometric edge features;
- using predicted edge scores to guide route construction;
- adding node penalties in the final version;
- keeping local search improvements after the learning-guided construction.

The second strategy was therefore not a fully neural solver, but a hybrid approach that combines machine learning with classical heuristics. This is important for the interpretation of the results. The method showed that learned information can be integrated into the route construction process, but the results must still be compared objectively with the classical heuristic baseline.

Chapter 5

Experimental Results and Discussion

The two implemented strategies were evaluated on the same set of mTSP benchmark instances in order to obtain a fair comparison. The evaluation focused on both the quality of the generated solutions and the computational effort required by each method. For this reason, the main metrics used in the comparison were the total cost of all routes, the cost of the longest route, the runtime, and the validity of the generated solution.

The total cost represents the sum of the distances travelled by all salesmen. This metric is important because it shows the overall efficiency of the solution. A lower total cost means that the algorithm generated routes with a smaller combined travelling distance. However, for the Multiple Traveling Salesman Problem, the total cost alone is not always enough to describe the quality of a solution. A solution may have a good total cost, but still assign one salesman a route that is much longer than the others.

For this reason, the longest route cost was also included in the evaluation. This metric is useful for measuring how balanced the routes are between salesmen. In practical scenarios, a balanced solution can be more desirable because the workload is distributed more evenly. Therefore, the comparison between the two strategies considered both global efficiency and route balance.

The first strategy produced valid solutions for all tested instances. This confirms that the classical heuristic approach was reliable from the point of view of feasibility. The method was able to assign all nodes to salesmen, construct complete routes, and ensure that every non-depot node was visited exactly once. The results also showed that the combination of geometric sweep assignment, nearest insertion, 2-opt local search, and inter-route relocation was strong enough to generate competitive solutions.

The second strategy also produced valid solutions for the tested instances. This is important because the learning-guided method modifies the route construction process using predicted information, but the final solution still needs to respect all mTSP constraints. The validation step confirmed that the hybrid approach was able to generate feasible routes while using the neural model as guidance.

When comparing the total cost, the classical heuristic strategy obtained slightly better results on average. The difference between the two strategies was not very large, but the learning-guided strategy did not consistently improve the total travelling cost. This shows that the first strategy remained a strong baseline and that the learned guidance was not sufficient to clearly outperform it for this metric.

However, the results for the longest route cost showed a different aspect of the comparison. The second strategy obtained better average performance for this metric, which suggests that the learning-guided method was sometimes able to create more balanced routes. This does not mean that it was better in every case, but it indicates that the additional information produced by the model may have helped the route construction process in some situations.

Runtime was also measured during the evaluation. The results showed that the second strategy had a lower average runtime in the final comparison. This can be explained by the use of candidate edges and by the way the final solver integrates the predictions during construction. However, runtime results should be interpreted carefully, because they can depend on implementation details, instance size, and the exact configuration used during testing.

Overall, the experimental results show that both strategies were functional and produced valid solutions. The first strategy was simpler, more direct, and slightly better in terms of average total cost. The second strategy was more experimental and did not consistently outperform the classical heuristic, but it was competitive and showed better average results for the longest route cost. This suggests that the learning-guided approach had some potential for improving route balance, even if it did not clearly improve the main cost objective.

The comparison also shows that the classical heuristic should not be considered only a weak baseline. In this project, it was improved through several local search techniques and therefore represented a strong reference method. The learning-guided strategy had to compete against a baseline that was already optimized with 2-opt and relocation moves. This makes the results more realistic, because the second strategy was compared with a practical heuristic and not with a very simple random or greedy method.

The main conclusion from the experiments is that the learning-guided strategy was competitive, but not clearly superior. The project demonstrates that machine learning can be integrated into a heuristic mTSP solver, but the obtained results do not support the claim that this approach is always better than the classical method. A more extensive training dataset, stronger model architecture, or additional optimization steps would probably be necessary in order to obtain more consistent improvements.

Table 5.1: Average results obtained by the two strategies based on the final evaluation

Strategy	Average total cost	Average longest route cost	Average runtime (s)
Strategy 1	3172.51	1194.13	1.108
Strategy 2	3186.54	1169.62	0.897

Chapter 6

Conclusions and Comparison with Related Work

The purpose of this project was to implement and compare two different strategies for solving the Multiple Traveling Salesman Problem. The first strategy was based on classical heuristics, while the second strategy used a learning-guided approach in which a neural model provided additional information during route construction. Both strategies were tested on the same benchmark instances and evaluated using total cost, longest route cost, runtime, and solution validity.

The first strategy proved to be a reliable baseline. It used geometric sweep assignment to divide the nodes between salesmen, nearest insertion to construct the routes, and local search techniques such as 2-opt and inter-route relocation to improve the initial solution. This approach is simple compared to more advanced metaheuristics, but it produced valid and competitive results for all tested instances. Its main advantage is that it is easier to understand, easier to debug, and does not require a training phase.

The second strategy was developed as an experimental hybrid method. Instead of directly generating the final routes using a neural network, the model was used to guide a heuristic solver. The strategy relied on candidate edges, predicted edge scores, and node-level penalties in order to modify the route construction process. This approach is related to modern learning-based optimization methods, where a model is used to estimate useful edges or guide the search, while the final solution is still obtained through a constructive or local search procedure.

The obtained results show that the learning-guided strategy was competitive, but not clearly superior to the classical heuristic. In terms of average total cost, the first strategy obtained slightly better results. In terms of average longest route cost, the second strategy performed better, which suggests that the learned guidance may have helped create more balanced routes in some cases. However, the improvements were not consistent enough to conclude that the second strategy is generally better.

This result is important because the first strategy was not a weak baseline. It already included several improvement steps, such as 2-opt and relocation moves between routes. Therefore, the learning-guided strategy had to compete with a practical heuristic that was already able to generate good solutions. The comparison shows that adding machine learning to a combinatorial optimization problem does not automatically guarantee better results, especially when the available training data and model complexity are limited.

Compared with related work, this project is closer to a hybrid heuristic approach than to a complete neural solver. In the literature, the Multiple Traveling Salesman Problem is usually treated as a generalization of the Traveling Salesman Problem, where several salesmen must visit all cities and return to the depot while minimizing a travel cost. Some variants also focus on minimizing the longest route in order to obtain a better workload balance between salesmen.

This is similar to the evaluation used in this project, where both total cost and longest route cost were considered.

Classical approaches for mTSP and TSP-like problems often use constructive heuristics followed by local search or neighborhood-based improvements. The first strategy follows this direction by combining a geometric assignment method with nearest insertion, 2-opt, and relocation moves. More advanced methods from the literature may use stronger neighborhoods, repeated perturbations, variable neighborhood search, or exact solvers, but the general idea of improving an initial feasible solution through local changes is also present in this project.

The second strategy is conceptually related to recent learning-based approaches for routing problems. In several modern methods, a neural model is trained to predict useful edges, edge probabilities, or heatmaps, and this information is then used to guide a search algorithm. The method implemented in this project follows a similar idea at a smaller scale: the model does not solve the problem alone, but it influences the heuristic by changing the attractiveness of candidate edges and by adding node-level penalties.

However, the comparison with specialized methods must be made carefully. This project did not include a full comparison against state-of-the-art mTSP solvers, exact optimization tools, or advanced neural architectures. The experiments were limited to the implemented strategies and to a relatively small set of benchmark instances. For this reason, the results should be interpreted mainly as an internal comparison between the two developed methods, not as proof that either method is competitive with the best existing algorithms.

The main limitation of the project is the limited scale of the experimental setup. The benchmark set contains a small number of instances, and the learning-guided strategy depends on training data generated by a heuristic teacher rather than by exact optimal solutions. Another limitation is that the neural model architecture is relatively simple compared to more advanced graph-based models used in current research. In addition, the hyperparameters and training process were not explored exhaustively, so better configurations may exist.

Future improvements could focus on extending the dataset, testing the methods on more benchmark instances, and comparing the results with stronger reference solvers. The learning-guided strategy could also be improved by using a graph neural network, better training labels, or a more advanced integration between the model predictions and the local search process. Another useful direction would be to optimize separately for total cost and longest route cost, since the experiments showed that the two objectives may behave differently.

In conclusion, the project successfully implemented and evaluated two different methods for solving the Multiple Traveling Salesman Problem. The classical heuristic strategy remained the more reliable method for minimizing average total cost, while the learning-guided strategy showed potential for improving route balance. The final results are mixed, but they are useful because they show both the strengths and the limitations of combining machine learning with classical heuristic optimization.